

PZ-HC05 蓝牙串口模块开发手册

本手册我们将向大家介绍 PZ-HC 蓝牙模块及其在普中 F407 最小系统&天马&麒麟开发板上的使用。本手册我们将使用 PZ-HC05 蓝牙串口模块实现蓝牙串口通信，并和手机连接，实现手机控制开发板。本章分为如下几部分内容：

- 1 PZ-HC05 蓝牙串口模块介绍
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-HC05 蓝牙串口模块介绍

1.1 特性参数

PZ-HC05 是一款高性能的主从一体蓝牙串口模块，可以同各种带蓝牙功能的电脑、蓝牙主机、手机、PDA、PSP 等智能终端配对，该模块支持非常宽的波特率范围：4800~1382400，并且兼容 5V 或 3.3V 单片机系统，可以很方便与您的产品进行连接，使用非常灵活、方便。该模块各参数如下所示：

项目	说明
接口特性	TTL，兼容 3.3V/5V 单片机系统
支持波特率	4800、9600（默认）、19200、38400、57600、115200、230400、460800、921600、1382400
其他特性	主从一体，指令切换，默认为从机。带状态指示灯，带配对状态输出
通信距离	10M（空旷地）
工作温度	-25℃~75℃
模块尺寸	20.37mm*38.84mm
工作电压	DC3.3V~5.0V
工作电流	配对中：30~40mA； 配对完毕未通信：1~8mA；通信中：5~20mA
Voh	3.3V@VCC=3.3V 3.7V@VCC=5.0V
Vol	0.4V(Max)
Vih	2.4V(Min)
Vil	0.4V(Max)

注意：通信中的电流和你的串口通信频繁程度成正比，如果单位时间内的数据通信量越大，电流则越高；反之，单位时间内的数据通信量越小，电流则越低（接近配对未通信的电流）。

1.2 模块说明

1.2.1 模块引脚说明

PZ-HC05 模块非常小巧（20.37mm*38.84mm），模块通过 1 个 2*4 的间距为 2.54 的排针与外部连接，模块外观如图所示：



从图中可以看到，从右到左依次为模块引出的 8 个脚，其中 NC 代表未与模块相连即待扩展脚，可用的管脚只有 6 个，各引脚的详细描述如图所示：

管脚名称	功能说明
VCC	电源（3.3V~5.0V）
GND	地
TXD	模块串口发送脚（TTL 电平，不能直接接 RS232 电平！），可接单片机的 RXD
RXD	模块串口接收脚（TTL 电平，不能直接接 RS232 电平！），可接单片机的 TXD
SET	用于进入 AT 状态；高电平有效（悬空默认为低电平）
STA	配对状态输出；配对成功输出高电平，未配对则输出低电平

另外，模块自带了一个状态指示灯：STA。该灯有 3 种状态，分别为：

1, 在模块上电的同时（也可以是之前），将 SET 脚 设置为高电平（接 VCC），此时 STA 慢闪（1 秒亮 1 次），模块进入 AT 状态，且此时波特率固定为 38400。

2, 在模块上电的时候，将 SET 脚 悬空或接 GND，此时 STA 快闪（1 秒 2 次），表示模块进入可配对状态。如果此时将 SET 脚 再拉高，模块也会进入 AT 状态，但是 STA 依旧保持快闪。

3, 模块配对成功，此时 STA 双闪（一次闪 2 下，2 秒闪一次）。

有了 STA 指示灯，我们就可以很方便的判断模块的当前状态，方便大家使用。

PZ-HC05 蓝牙串口模块所有功能都是通过 AT 指令集控制，比较简单，该部分使用的详细信息，请参考 HC05 蓝牙指令集.pdf。

使用 PZ-HC05 蓝牙串口模块，任何单片机（3.3V/5V 电源）都可以很方便的实现蓝牙通信，从而与包括电脑、手机、平板电脑等各种带蓝牙的设备连接。PZ-HC05 蓝牙串口模块的原理图如图所示：

比如我们要查看模块的版本：

串口发送： AT+VERSION?\r\n

模块回应： +VERSION:2.0-20100601

OK

3. 常用指令说明及测试

注意，这里我们通过将模块连接电脑串口，来测试模块的指令，注意模块不能和 RS232 串口直连。

①修改模块主从指令

AT+ROLE=0 或 1, 该指令来设置模块为从机或主机，并且可以通过 AT+ROLE? 来查看模块的主从状态，如图所示：



我们模块出厂默认设置为从机，所以发送 AT+ROLE?，得到的返回值为：+ROLE:0，发送 AT+ROLE=1，即可设置模块为主机，设置成功模块返回 OK 作为应答。注意串口调试助手要勾选发送新行，这样就会自动发送回车了。

②设置记忆指令

AT+CMODE=1， 该指令设置模块可以对任意地址的蓝牙模块进行配对， 模块默认设置为该参数。AT+CMODE=0， 该指令设置模块为指定地址配对，如果先设置模块为任意地址，然后配对，接下去使用该指令，则模块会记忆最后一次配对

的地址，下次上电会一直搜索该地址的模块，直到搜索到为止。

③修改通信波特率指令

AT+UART=<Param1>,<Param2>,<Param3>, 该指令用于设置串口波特率、停止位、校验位等。Param1 为波特率,可选范围为：4800、9600、19200、38400、57600、115200、230400、460800、921600、1382400； Param2 为停止位选择，0 表示 1 位停止位，1 表示 2 位停止位； Param3 为校验位选择，0 表示没有校验位（None），1 表示奇校验（Odd），2 表示偶校验（Even）。

比如我们发送：AT+UART=9600,0,0, 则是设置通信波特率为 9600，1 位停止位，没有校验位，这也是我们模块的默认设置。

④修改密码指令

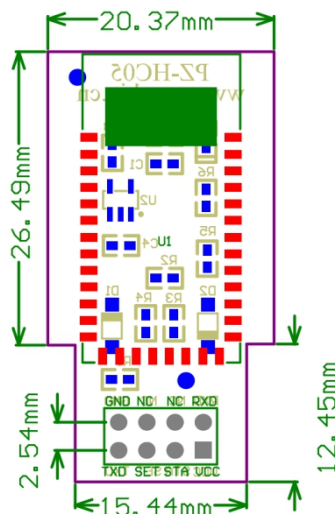
AT+PSWD=<password>, 该指令用于设置模块的配对密码，password 必须为 4 个字节长度。

⑤修改蓝牙模块名字

AT+NAME=<name>, 该指令用于设置模块的名字，name 为你要设置的名字，必须为 ASCII 字符，且最长不能超过 32 个字符。模块默认的名字为 HC05。比如发送：AT+NAME=PZ-HC05, 即可设置模块名字为“PZ-HC05”。

1.3 结构尺寸

PZ-HC05 蓝牙模块尺寸结构如下图所示：



2 硬件设计

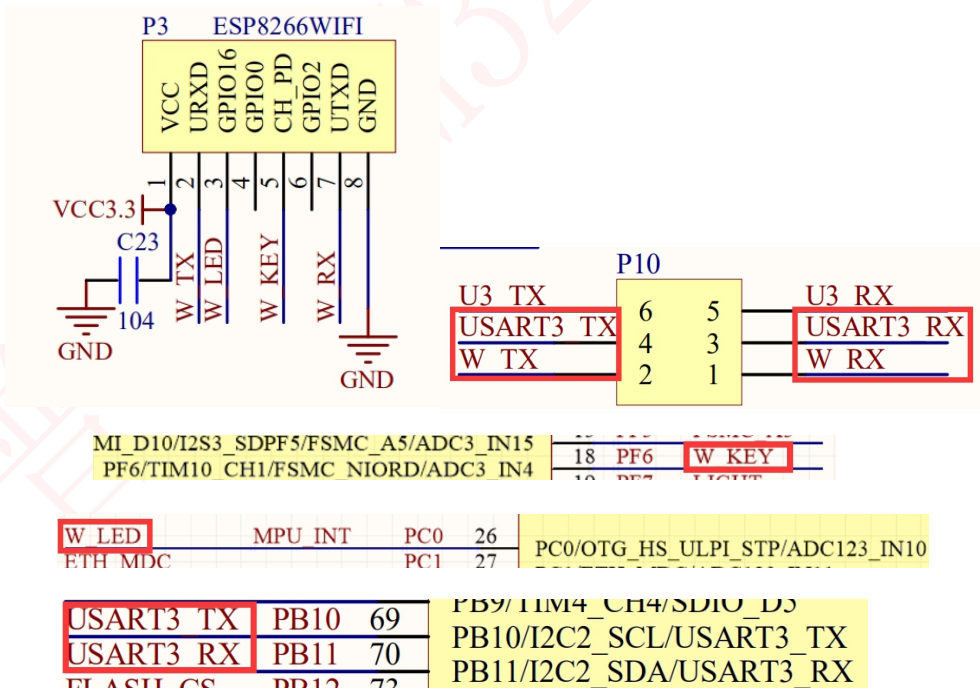
2.1 硬件准备

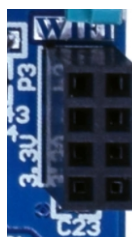
本实验所需要的硬件资源如下：

- ①F407 最小系统&天马&麒麟开发板 1 个
- ②TFTLCD 模块（不用彩屏也可用串口输出）
- ③PZ-HC05 蓝牙模块 1 个
- ④USB 线一条（用于供电和模块与电脑串口调试助手通信）
- ⑤带蓝牙功能的安卓手机一台

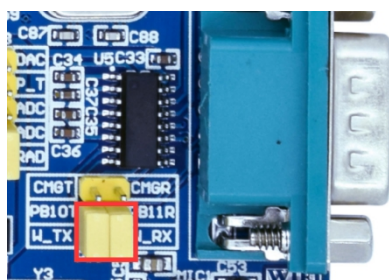
2.2 模块与开发板连接

PZ-HC05 模块可直接与天马&麒麟开发板板载的 WIFI&蓝牙模块接口（P3）进行连接，该接口与 MCU 连接原理图（以麒麟为例）如下图所示：





从上图看出,WIFI 接口与 STM32 使用时,必须将 P10 的 USART3_TX(PB10 和 W_TX 以及 USART3_RX (PB11) 和 W_RX 连接,才能完成和 STM32 的连接。默认天马&麒麟的 P10 端子上有黄色跳线帽,直接短接到对应端即可。如下图所示:



对于 F407 最小系统,是没有 WIFI 接口的,所以需要通过杜邦线将模块管脚与上述 STM32 的 IO 口连接即可。

PZ-HC05 模块与 STM32 的 IO 口连接关系如下: (PZ-HC05→STM32F407)

VCC→3.3V
GND→GND
RXD→PB10
TXD→PB11
SET→PF6
STA→PC0

注意: 使用天马&麒麟开发板连接模块时,模块插入 WIFI 接口处请注意方向,可将模块电源与 WIFI 接口丝印电源对应即可。

3 软件设计

本实验所实现的功能为: 开机检测 PZ-HC05 蓝牙模块是否存在,如果检测不成功,则报错。检测成功之后,显示模块的主从状态,并显示模块是否处于连接状态,DS0 指示灯闪烁提示程序运行正常。按 KEY1 按键,可以开启/关闭自动发送数据(通过蓝牙模块发送);按 KEY_UP 按键可以切换模块的主从状态。蓝

牙模块接收到的数据，将直接显示在 LCD 上（仅支持 ASCII 字符显示）。结合手机端蓝牙软件（蓝牙串口助手 V0.16.apk），可以实现手机无线控制开发板（点亮和关闭 DS1 指示灯）。

3.1 usart3.c

我们打开本实验工程，可以看到我们的工程 APP 列表中多了 usart3.c 和 hc05.c 源文件，以及头文件 usart3.h、hc05.h。首先，我们来看 usart3.c 里面代码，如下：

```
1.  #include "usart3.h"
2.  #include "stdarg.h"
3.  #include "stdio.h"
4.  #include "string.h"
5.  #include "time.h"
6.
7.
8.  //串口接收缓存区
9.  u8 USART3_RX_BUF[USART3_MAX_RECV_LEN]; //接收缓冲,最大 USART3_MAX_RECV_LEN 个字节.
10. u8 USART3_TX_BUF[USART3_MAX_SEND_LEN]; //发送缓冲,最大 USART3_MAX_SEND_LEN 字节
11.
12.
13. //通过判断接收连续 2 个字符之间的时间差不大于 10ms 来决定是不是一次连续的数据.
14. //如果 2 个字符接收间隔超过 10ms,则认为不是 1 次连续数据.也就是超过 10ms 没有接收到
15. //任何数据,则表示此次接收完毕.
16. //接收到的数据状态
17. //[15]:0,没有接收到数据;1,接收到了一批数据.
18. //[14:0]:接收到的数据长度
19. u16 USART3_RX_STA=0;
20.
21. void USART3_Init(u32 bound)
22. {
23.
24.     GPIO_InitTypeDef GPIO_InitStructure;
25.     USART_InitTypeDef USART_InitStructure;
26.     NVIC_InitTypeDef NVIC_InitStructure;
27.
28.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB,ENABLE); //使能 GPIOA 时钟
29.     RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART3,ENABLE); //使能 USART3 时钟
30.
31.     //串口 3 对应引脚复用映射
32.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource10,GPIO_AF_USART3);
```

```

33.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource11,GPIO_AF_USART3);
34.
35.     //USART3 端口配置
36.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10|GPIO_Pin_11 ;
37.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;//复用功能
38.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;    //速度 50MHz
39.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽复用输出
40.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉
41.     GPIO_Init(GPIOB,&GPIO_InitStructure);
42.
43.     //Usart3 NVIC 配置
44.     NVIC_InitStructure.NVIC_IRQChannel = USART3_IRQn;
45.     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=2 ;//抢占优先级 0
46.     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3;    //子优先级 0
47.     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;    //IRQ 通道使能
48.     NVIC_Init(&NVIC_InitStructure); //根据指定的参数初始化 VIC 寄存器
49.
50.     //USART3 初始化设置
51.     USART_InitStructure.USART_BaudRate = bound;//串口波特率
52.     USART_InitStructure.USART_WordLength = USART_WordLength_8b;//字长为 8 位数据格式
53.     USART_InitStructure.USART_StopBits = USART_StopBits_1;//一个停止位
54.     USART_InitStructure.USART_Parity = USART_Parity_No;//无奇偶校验位
55.     USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;//无硬件
    数据流控制
56.     USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式
57.     USART_Init(USART3, &USART_InitStructure); //初始化串口 3
58.
59.     USART_ITConfig(USART3, USART_IT_RXNE, ENABLE);//开启串口接受中断
60.
61.     USART_Cmd(USART3, ENABLE);    //使能串口 3
62.
63.     TIM7_Int_Init(100-1,8400-1);    //10ms 中断一次
64.     USART3_RX_STA=0;    //清零
65.     TIM_Cmd(TIM7,DISABLE);    //关闭定时器 7
66. }
67.
68. void USART3_IRQHandler( void )
69. {
70.     u8 res;
71.     if(USART_GetITStatus(USART3, USART_IT_RXNE) != RESET)//接收到数据
72.     {
73.         res =USART_ReceiveData(USART3);
74.         if((USART3_RX_STA&(1<<15))==0)//接收完的一批数据,还没有被处理,则不再接收其他数据
75.         {

```

```

76.         if(USART3_RX_STA<USART3_MAX_RECV_LEN)    //还可以接收数据
77.         {
78.             TIM_SetCounter(TIM7,0);//计数器清空                //计数器清空
79.             if(USART3_RX_STA==0)                    //使能定时器 7 的中断
80.             {
81.                 TIM_Cmd(TIM7,ENABLE);//使能定时器 7
82.             }
83.             USART3_RX_BUF[USART3_RX_STA++]=res; //记录接收到的值
84.         }
85.         else
86.         {
87.             USART3_RX_STA|=1<<15;                //强制标记接收完成
88.         }
89.     }
90. }
91. }
92.
93.
94. //串口 3,printf 函数
95. //确保一次发送数据不超过 USART3_MAX_SEND_LEN 字节
96. void usart3_printf(char* fmt,...)
97. {
98.     u16 i,j;
99.     va_list ap;
100.    va_start(ap,fmt);
101.    vsprintf((char*)USART3_TX_BUF,fmt,ap);
102.    va_end(ap);
103.    i=strlen((const char*)USART3_TX_BUF);        //此次发送数据的长度
104.    for(j=0;j<i;j++)                            //循环发送数据
105.    {
106.        while(USART_GetFlagStatus(USART3,USART_FLAG_TC)==RESET); //循环发送,直到发送完毕
107.        USART_SendData(USART3,USART3_TX_BUF[j]);
108.    }
109. }

```

这部分代码主要实现了串口 3 的初始化,以及实现了串口 3 的 printf 函数: u3_printf, 和串口 3 的接收处理。串口 3 的数据接收,采用了定时判断的方法,对于一次连续接收的数据,如果出现连续 10ms 没有接收到任何数据,则表示这次连续接收数据已经结束。

Usart3.h 里面的代码我们就不在这里列出了,请大家参考对应源码。

3.2 hc05.c

下面我们来看下 hc05.c 里面的代码，如下：

```
1.  #include "SysTick.h"
2.  #include "usart.h"
3.  #include "usart3.h"
4.  #include "hc05.h"
5.  #include "led.h"
6.  #include "string.h"
7.  #include "math.h"
8.
9.
10.
11.
12.  //初始化 HC05 模块
13.  //返回值:0,成功;1,失败.
14.  u8 HC05_Init(void)
15.  {
16.      u8 retry=10,t;
17.      u8 temp=1;
18.
19.      GPIO_InitTypeDef GPIO_InitStructure;
20.
21.      RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOF|RCC_AHB1Periph_GPIOC, ENABLE);// 使 能
      GPIOC,GPIOF 时钟
22.
23.      GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0; //LED 对应引脚
24.      GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN; //普通输入模式
25.      GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; //100M
26.      GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉
27.      GPIO_Init(GPIOC, &GPIO_InitStructure); //初始化 GPIOC0
28.
29.
30.      GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6; //KEY 对应引脚
31.      GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //普通输出模式
32.      GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽输出
33.      GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; //100MHz
34.      GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉
35.      GPIO_Init(GPIOF, &GPIO_InitStructure); //根据设定参数初始化 PF6
36.
37.      GPIO_SetBits(GPIOF,GPIO_Pin_6);
38.
39.      HC05_KEY=1;
```

```

40.     HC05_LED=1;
41.
42.     USART3_Init(9600); //初始化串口 2 为:9600,波特率.
43.
44.     while(retry--)
45.     {
46.         HC05_KEY=1; //KEY 置高,进入 AT 模式
47.         delay_ms(10);
48.         usart3_printf("AT\r\n"); //发送 AT 测试指令
49.         HC05_KEY=0; //KEY 拉低,退出 AT 模式
50.         for(t=0;t<10;t++) //最长等待 50ms,来接收 HC05 模块的响应
51.         {
52.             if(USART3_RX_STA&0X8000)break;
53.             delay_ms(5);
54.         }
55.         if(USART3_RX_STA&0X8000) //接收到一次数据了
56.         {
57.             temp=USART3_RX_STA&0X7FFF; //得到数据长度
58.             USART3_RX_STA=0;
59.             if(temp==4&&USART3_RX_BUF[0]=='O'&&USART3_RX_BUF[1]=='K')
60.             {
61.                 temp=0;//接收到 OK 响应
62.                 break;
63.             }
64.         }
65.     }
66.     if(retry==0)temp=1; //检测失败
67.     return temp;
68. }
69. //获取 HC05 模块的角色
70. //返回值:0,从机;1,主机;0XFF,获取失败.
71. u8 HC05_Get_Role(void)
72. {
73.     u8 retry=0X0F;
74.     u8 temp,t;
75.     while(retry--)
76.     {
77.         HC05_KEY=1; //KEY 置高,进入 AT 模式
78.         delay_ms(10);
79.         usart3_printf("AT+ROLE?\r\n"); //查询角色
80.         for(t=0;t<20;t++) //最长等待 200ms,来接收 HC05 模块的响应
81.         {
82.             delay_ms(10);
83.             if(USART3_RX_STA&0X8000)break;

```

```

84.     }
85.     HC05_KEY=0;           //KEY 拉低,退出 AT 模式
86.     if(USART3_RX_STA&0X8000) //接收到一次数据了
87.     {
88.         temp=USART3_RX_STA&0X7FFF; //得到数据长度
89.         USART3_RX_STA=0;
90.         if(temp==13&&USART3_RX_BUF[0]=='+')//接收到正确的应答了
91.         {
92.             temp=USART3_RX_BUF[6]-'0';//得到主从模式值
93.             break;
94.         }
95.     }
96. }
97. if(retry==0)temp=0XFF;//查询失败.
98. return temp;
99. }
100. //HC05 设置命令
101. //此函数用于设置 HC05,适用于仅返回 OK 应答的 AT 指令
102. //atstr:AT 指令串.比如:"AT+RESET"/"AT+UART=9600,0,0"/"AT+ROLE=0"等字符串
103. //返回值:0,设置成功;其他,设置失败.
104. u8 HC05_Set_Cmd(u8* atstr)
105. {
106.     u8 retry=0X0F;
107.     u8 temp,t;
108.     while(retry--)
109.     {
110.         HC05_KEY=1;           //KEY 置高,进入 AT 模式
111.         delay_ms(10);
112.         usart3_printf("%s\r\n",atstr); //发送 AT 字符串
113.         HC05_KEY=0;           //KEY 拉低,退出 AT 模式
114.         for(t=0;t<20;t++)      //最长等待 100ms,来接收 HC05 模块的回应
115.         {
116.             if(USART3_RX_STA&0X8000)break;
117.             delay_ms(5);
118.         }
119.         if(USART3_RX_STA&0X8000) //接收到一次数据了
120.         {
121.             temp=USART3_RX_STA&0X7FFF; //得到数据长度
122.             USART3_RX_STA=0;
123.             if(temp==4&&USART3_RX_BUF[0]=='0')//接收到正确的应答了
124.             {
125.                 temp=0;
126.                 break;
127.             }

```

```

128.     }
129. }
130.     if(retry==0)temp=0xFF;//设置失败.
131.     return temp;
132. }

```

此部分代码总共 3 个函数：

1, HC05_Init 函数，该函数用于初始化与 PZ-HC05 连接的 IO 口，并通过 AT 指令检测 PZ-HC05 蓝牙模块是否已经连接。

2, HC05_Get_Role 函数，该函数用于获取 PZ-HC05 蓝牙模块的主从状态，这里利用 AT+ROLE?指令获取模块的主从状态。

3, HC05_Set_Cmd 函数，该函数是一个 PZ-HC05 蓝牙模块的通用设置指令，通过调用该函数，可以方便的修改 PZ-HC05 蓝牙串口模块的各种设置。

hc05.h 里面的代码我们也不列出了，请大家参考对应源码。

3.3 main.c

最后我们打开 main.c 文件，里面的代码如下：

```

1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "key.h"
5.  #include "usart.h"
6.  #include "tftlcd.h"
7.  #include "hc05.h"
8.  #include "usart3.h"
9.  #include "string.h"
10.
11.
12.  //显示 HC05 模块的主从状态
13.  void HC05_Role_Show(void)
14.  {
15.      if(HC05_Get_Role()==1)LCD_ShowString(10,140,200,16,16,"ROLE:Master"); //主机
16.      else LCD_ShowString(10,140,200,16,16,"ROLE:Slave "); //从机
17.  }
18.
19.  //显示 HC05 模块的连接状态
20.  void HC05_Sta_Show(void)
21.  {
22.      if(HC05_LED)LCD_ShowString(110,140,120,16,16,"STA:Connected "); //连接成功

```



```

23.         else LCD_ShowString(110,140,120,16,16,"STA:Disconnect"); // 未连
           接
24.     }
25.
26.
27.     int main()
28.     {
29.         u8 t=0;
30.         u8 key;
31.         u8 sendmask=0;
32.         u8 sendcnt=0;
33.         u8 sendbuf[20];
34.         u8 reclen=0;
35.
36.         SysTick_Init(168);
37.         NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
38.         LED_Init();
39.         KEY_Init();
40.         USART1_Init(115200);
41.         TFTLCD_Init(); //LCD 初始化
42.
43.
44.         FRONT_COLOR=RED;
45.         LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"PRECHIN");
46.         LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.com");
47.         LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,16,"BT05 BlueTooth Test");
48.         delay_ms(1000); //等待蓝牙模块上电稳定
49.
50.         while(HC05_Init()) //初始化 HC05 模块
51.         {
52.             LCD_ShowString(10,90,200,16,16,"HC05 Error! ");
53.             delay_ms(500);
54.             LCD_ShowString(10,90,200,16,16,"Please Check!!!");
55.             delay_ms(100);
56.         }
57.         LCD_ShowString(10,90,210,16,16,"KEY_UP:ROLE KEY1:SEND/STOP");
58.         LCD_ShowString(10,110,200,16,16,"HC05 Standby!");
59.         LCD_ShowString(10,160,200,16,16,"Send:");
60.         LCD_ShowString(10,180,200,16,16,"Receive:");
61.
62.         FRONT_COLOR=BLUE;
63.         HC05_Role_Show();
64.         delay_ms(100);
65.         USART3_RX_STA=0;

```

```

66.     while(1)
67.     {
68.         key=KEY_Scan(0);
69.         if(key==KEY_UP_PRESS)                //切换模块主从设置
70.         {
71.             key=HC05_Get_Role();
72.             if(key!=0XFF)
73.             {
74.                 key=!key;                //状态取反
75.                 if(key==0)HC05_Set_Cmd("AT+ROLE=0");
76.                 else HC05_Set_Cmd("AT+ROLE=1");
77.                 HC05_Role_Show();
78.                 HC05_Set_Cmd("AT+RESET");    //复位 HC05 模块
79.                 delay_ms(200);
80.             }
81.         }
82.         else if(key==KEY1_PRESS)
83.         {
84.             sendmask=!sendmask;          //发送/停止发送
85.             if(sendmask==0)LCD_Fill(10+40,160,240,160+16,WHITE); //清除显示
86.         }
87.         else delay_ms(10);
88.         if(t==50)
89.         {
90.             if(sendmask)                //定时发送
91.             {
92.                 sprintf((char*)sendbuf,"PREHICN HC05 %d\r\n",sendcnt);
93.                 LCD_ShowString(10+40,160,200,16,16,sendbuf);    //显示发送数据
94.                 usart3_printf("PREHICN HC05 %d\r\n",sendcnt);    //发送到蓝牙模块
95.                 sendcnt++;
96.                 if(sendcnt>99)sendcnt=0;
97.             }
98.             HC05_Sta_Show();
99.             t=0;
100.            LED1=!LED1;
101.        }
102.        if(USART3_RX_STA&0X8000)        //接收到一次数据了
103.        {
104.            LCD_Fill(10,200,240,320,WHITE); //清除显示
105.            reclen=USART3_RX_STA&0X7FFF;    //得到数据长度
106.            USART3_RX_BUF[reclen]='\0';    //加入结束符
107.            printf("reclen=%d\r\n",reclen);
108.            printf("USART3_RX_BUF=%s\r\n",USART3_RX_BUF);
109.            if(reclen==10||reclen==11)    //控制 D2 检测

```

```

110.      {
111.          if(strcmp((const char*)USART3_RX_BUF, "+LED2 ON\r\n")==0)LED2=0;//打开 LED2
112.          if(strcmp((const char*)USART3_RX_BUF, "+LED2 OFF\r\n")==0)LED2=1;//关闭 LED2
113.      }
114.      LCD_ShowString(10,200,209,119,16,USART3_RX_BUF);//显示接收到的数据
115.      USART3_RX_STA=0;
116.  }

117.      t++;
118.  }
119. }

```

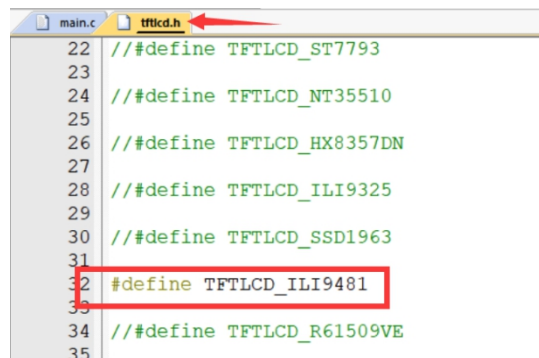
此部分代码比较简单，主要是调用前面编写的函数，加上额外的显示控制等，这里我们就不多说，大家可以查看源码。

4 实验现象

首先，请先确保硬件都已经连接好：

- ①PZ-HC05 模块与 F407 最小系统&天马&麒麟开发板连接（可参考硬件设计小节）
- ②开发板上插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口输出。
- ③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）LCD 界面显示如下图所示：

```
PRECHIN
www.prechin.com
BT05 BlueTooth Test

KEY_UP:ROLE  KEY1:SEND/STO
HC05 Standby!

ROLE:Slave  STA:Disconnect
Send:
Receive:
```

可以看到，此时模块的状态是从机（Slave），未连接（Disconnect）。发送和接收区都没有数据，同时蓝牙模块的 STA 指示灯快闪（1 秒 2 次），表示模块进入可配对状态，目前尚未连接。

下面我们来演示一下 PZ-HC05 蓝牙串口模块与手机蓝牙连接，这里我们先设置蓝牙模块为从机（Slave）角色，以便和手机连接。然后在手机上安装蓝牙串口助手 V0.16.apk 这个软件，该软件可在该模块资料目录内的“调试工具”找到。安装完软件后，我们打开该软件，进入搜索蓝牙设备界面，第一次连接蓝牙设备的时候会让你输入配对码，输入完成后一段时间就可以连接成功，如图所示：



从上图可以看出，手机已经搜索到我们的模块了：PZ-HC05（**这是我们通过 AT 指令修改了模块名称，出厂模块并非此名**），点击这个设备，即进入，点击连接设备，连接成功后，TFTLCD 上会显示“Connected”，并且手机界面如图所

示：

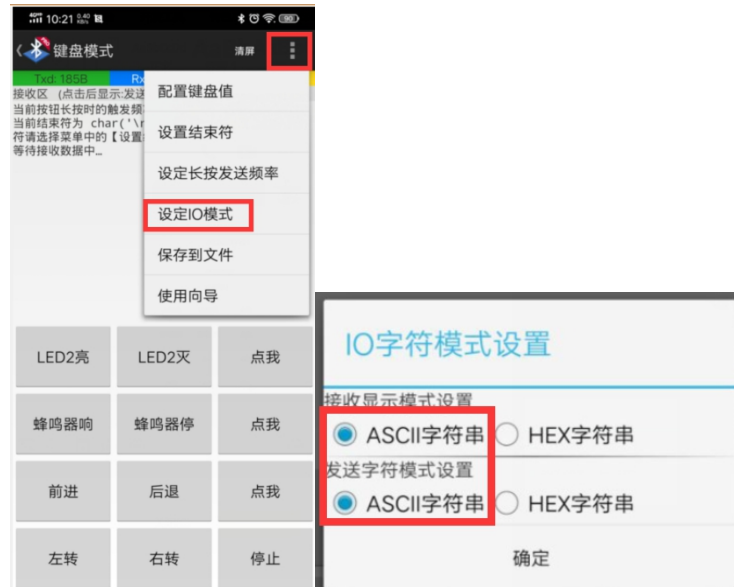


这几种模式都可以用来测试 PZ-HC05 蓝牙串口模块，这里我们以该 APP 操作较为复杂的键盘模式来演示，我们选择“键盘模式”，打开后界面如下：



从图中可以看到，有一个 LED2 亮和 LED2 灭按键，这个是我们自定义的，初始安装这个软件的时候并没有此功能按键，可以点击右上角的配置按钮，选择配置键值。操作如下：

首先我们先设定 IO 模式为 ASCII，如下所示：



然后选择“配置键盘值”，如下图所示：



然后点击“点我”即可对其配置，比如我们设置按键名称为“LED2 亮”，按下发送值为“+LED2 ON”，然后点击确定即可。如下图所示：



设置完成后再次点击右上角的配置按钮，选择保存键盘配置即可。如下图所示：



我们设置了两个按键功能，一个用于点亮 LED2，一个用于熄灭 LED2。

然后我们点击按钮就可以通过蓝牙发送对应的键值，PZ-HC05 蓝牙串口模块就会接收手机发送的数据并显示在 LCD 上，同时控制开发板上的 DS1 指示灯的亮灭。

大家如果需要使用手机通过蓝牙控制更多 STM32 外部设备，可以增加相应的按键功能即可。至此我们就介绍完 PZ-HC05 蓝牙串口模块的使用，大家稍作改进就可以通过 PZ-HC05 蓝牙串口模块做很多有意思的东西。

5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）